

# Lecture 13: Tic-Tac-Toe Logic & Introduction to APIs

## Project Breakdown: Tic-Tac-Toe Game:

### 1. Building the Win Logic:

A 3x3 grid can be represented using index positions 0 to 8:

|   |   |   |
|---|---|---|
| 0 | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

#### Win Conditions:

- Rows: 0-1-2, 3-4-5, 6-7-8
- Columns: 0-3-6, 1-4-7, 2-5-8
- Diagonals: 0-4-8, 2-4-6

### 2. Event Delegation & Turn Logic:

- Instead of attaching an event listener to each of the 9 buttons individually, a single listener is attached to the parent container (board.addEventListener('click', ...)).
- A variable tracks whose turn it is, toggling between 'X' and 'O' after each valid move.

```
if(turn == 'X')
  turn = 'O';
else
  turn = 'X';
```

### 3. Storing Moves in an Array:

To separate game logic from the visual DOM, a gridBox array of size 9 tracks the state of the board, initialized with empty strings.

```
const gridBox = ['', '', '', '', '', '', '', '', ''];
```

On each click, the index of the clicked box is extracted from its id, and that exact position in the array is updated with the current turn.

```
const index = box.id;
gridBox[index] = turn;
```

### 4. Checking for a Winner:

A checkWinner function checks all possible winning combinations for the current player.

```
function checkWinner(player) {
  if (gridBox[0] == player && gridBox[1] == player && gridBox[2] == player) {
    return true;
  }
  // similar checks for remaining rows, columns, and diagonals
  else {
    return false;
  }
}
```

After each move, this function is called for the player who just played. If it returns true, the winner is displayed.

## 5. Preventing Overwrites:

Once a box is filled, it should not be clickable again. Similarly, once the game is won or drawn, no further moves should be processed.

```
if (winnerDecided || totalInsert == 9 || gridBox[e.target.id] != "") {  
    return;  
}
```

## 6. Draw Condition:

- A counter (totalInsert) increments on every valid move.
- A draw occurs when all 9 boxes are filled (totalInsert === 9) and the checkWinner function has not found a winner.

## 7. Resetting the Game:

A reset function must return both the visual DOM and the internal JavaScript logic to their initial states simultaneously.

```
for(let i=0;i<9;i++){  
    document.getElementById(i).textContent = "";  
    gridBox[i] = "";  
}
```

```
totalInsert = 0;  
winnerDecided = false;  
stat.textContent = "Player X's turn";
```

**Note:** Since event delegation is used on the parent container, clicking on the edges or gaps of the parent (outside the actual boxes) will still register as a click on the parent, so this needs to be handled carefully.

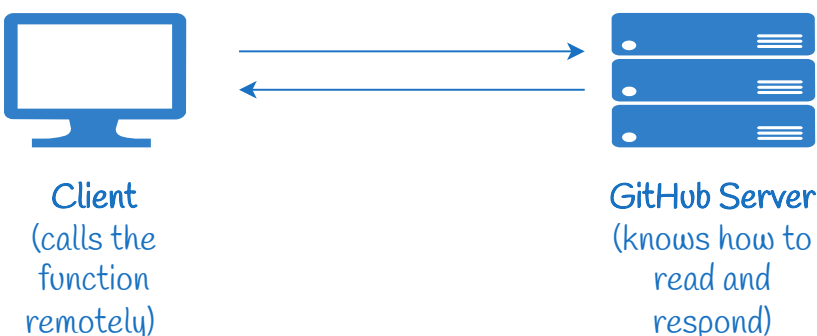
## Introduction to APIs:

### What is an API?

API stands for Application Programming Interface. At its core, an API is simply a way to call a function, just remotely, on someone else's server.

**Example:** [https://api.github.com/users?per\\_page=20](https://api.github.com/users?per_page=20)

This requests 20 users' worth of data from GitHub's server.



## Why APIs Exist?

APIs are used whenever the internal working of a system isn't known or accessible directly, but a defined interface is exposed to interact with it. Essentially, all CRUD operations happen through APIs. The conversation just happens at a higher, more abstracted level.

### API Keys:

An API key is a unique identifier (like a unique ID) assigned to a user, used to track and control their usage.

### Rate Limiting:

- 1 hour: rate limit of 100 requests
- Tracked based on the source IP address

**Note:** Since a user's IP can be manipulated, rate limiting by IP alone isn't fully reliable, hence the need for API keys tied to a specific user account.

### Free vs Paid API Keys:

- **Free Tier:** Generally comes with a trial period and a strictly limited request rate.
- **Paid Tier:** Offers significantly higher limits (e.g., 10,000 maximum requests). Paid keys must be kept secure, as misuse can lead to the key being revoked or deleted entirely.

## Why Rate Limits Exist?

A server (like GitHub's) doesn't have unlimited capacity. It can only serve data it already has, so requests are relatively lightweight. However, services like LLM models require heavy computation for every request, which is why strict, hard-coded limits are enforced on those APIs to control cost and load.